Engine Matrix EMS/VMS rev2e pack (rev2f notebooks) — Independent Strict Audit

Date: 2026-10-26
Scope: `ENGINE_MATRIX_EMS_VMS_REV2E_AUDIT_FOR_GPT_2026-10-26(1).zip`
Notebook SHA256 prefixes: EMS SAFE `c0ade787`, EMS RISKY `b39016d6`, VMS `b01992c5`.
Decision: **NO-GO for automatic winner adoption or production switch**. **GO for CPU repairs and rerun of negative tests**. After fixes, bounded GPU correctness/capacity smoke is reasonable. No GPU inference was run in this audit.

বাংলা নির্বাহী সারাংশ

এই ZIP-এর বর্তমান notebooks আসলে **rev2f**: EMS SAFE, EMS RISKY, VMS। README ও disposition পড়ার পরে source-level স্বাধীন পরীক্ষায় সাতটি **negative scenario**-তে acceptance detector-এর ফাঁক পাওয়া গেছে। বিশেষত **XL/canary** অনুপস্থিত, **oracle canary error**, এবং **PCIe-IPC backend initialize** না হলেও **winner** গ্রহণের পথ এখনও রয়েছে। তাই এই সংস্করণে `adoptable=True`-কে production safety certificate ধরা যাবে না।

প্রধান করণীয়: R1 (missing correctness evidence), R2 (no-first-token throughput), R6 (PCIe-backend verification) আগে বন্ধ করুন; R3/R4-এর heuristic/static-guard উন্নত করুন; R5-এর planned row count assert যোগ করুন; R7-এর `BOUNDARY`-কে production-approved নয়, **ARC task validation candidate** হিসেবে রাখুন। সংশ্লিষ্ট exact code-level patches ও negative tests নিচে দেওয়া আছে। সব upstream source সরাসরি পূর্ণ URL-এ দেওয়া হয়েছে যাতে Claude খুঁজতে পারে। এখানে কোনো **GPU run**, **Kaggle push** বা **production modification** করা হয়নি।

What was checked first

Read `README_FOR_GPT.md` and `notes/EMS_VMS_rev2e_independent_audits_disposition_2026-10-26.md` first. Then inspected `builders/build_ems.py`, `builders/build_vms.py`, `harness/e14_measure.py`, `harness/harness_ems_nb.py`, `harness/harness_vms_nb.py`, selected SGLang 0.5.21 and vLLM 0.32.0 pinned-source snapshots, `MANIFEST.txt` and all three generated notebooks. All three notebooks parse as JSON and all code cells compile syntactically. Their hashes match the ZIP manifest. The ZIP contains previous rev2c Kaggle CPU-kit reports and older harness logs; those by themselves do **not** prove that the final rev2f notebooks passed a fresh Kaggle run. The disposition describes newer self-checks, but a self-report is not a GPU result.

Existing useful fixes were preserved: integer token IDs in VMS; stream `[DONE]` plus `finish_reason`; fail-closed missing target oracle in the basic paths; post-divergence heuristic; bounded window cleanup; explicit measurement/correctness/capacity labels; PR #45151 atomic write plus arm skip if patch absent; per-arm exception isolation; ref-vs-target completeness; prefix-cache counters; XL diagnostics. The findings below are **additional defects and limitations** rather than a rehash of already-fixed rev2c items.

Highest-priority new findings (source-level, independently tested)

ID	Priority	File:lines	Failing negative test	Impact
R1	**BLOCKER**	`build_vms.py:727–739, :763–805`; `build_ems.py:681–711, :556–557`	Missing XL + missing VMS canary -> `adoptable=True`; missing EMS XL + oracle canary errors -> `_gated=True`	Required correctness tests can be absent while arm is marked safe

ID	Priority	File:lines	Failing negative test	Impact
R2	**HIGH**	`build_vms.py:415–441`	`no_first_token=3` alongside otherwise clean counters -> `vms_window_healthy=True`	No-first-token streams not disqualifying a benchmark
R3	**HIGH detector coverage**	`build_vms.py:475–491`, `build_ems.py:713–735`	`"1234567890"*5 + "!"*64` -> `vms_text_bad=False`	Any >=30% digits text bypasses degeneration even if dominated by corrupted punctuation; same limitation in EMS
R4	**HIGH guard correctness**	`build_vms.py:611–643`	Packed MTP tensors, empty ignore list and no normal `.weight` linear keys -> `vms_int4_mtp_static=True`	The static guard can declare checkpoint 'OK' without actually proving dense BF16 MTP exclusions
R5	**MEDIUM defensive**	`build_vms.py:516–547, :662–706`	Candidate and A/A both contain only 1 identical row -> `vms_judge=PASS`	Comparator never validates known intended 6 / full-concurrent / 4 XL prompt counts; ordinary generator does return the planned number, so this is a defensive edge case, not demonstrated in real run
R6	**HIGH evidence integrity**	`build_vms.py:820–824, :727–739`; `build_ems.py:670–679, :681–711`	PCIe-IPC requested but `initialized=False` -> VMS `adoptable=True`	Named PCIe optimization can be compared as if active even if backend silently falls back
R7	**HIGH release-policy limitation**	`build_vms.py:676–695, :727–739`; `build_ems.py:740–761, :1120–1122`	Nonexact `BOUNDARY` is intentionally `adoptable`; only degenerate continuations are rejected	A fluent but wrong continuation can evade the tail heuristic; BOUNDARY is not proof of task accuracy
R8	**MEDIUM practical planning**	`build_ems.py:287–291, :764–850`; three generated notebooks	SAFE and RISKY each create separate reference/oracle launches; previous 260-minute estimate was per unsplit matrix	Quota and startup cost of 3 independent notebooks must be recalculated; isolation is a valid reason for duplication

Evidence status

R1–R6 were reproduced with the actual functions extracted and compiled from the supplied builder strings, using synthetic row/metric/checkpoint inputs. R7 is a deliberate, documented acceptance policy, not an accidental implementation defect. R8 follows directly from the split notebook flow. **No item here proves the GPU has already emitted wrong ARC answers**.

Fix 1 — Missing XL/canary must not pass a certification gate

Current VMS code accepts `a.get("xl_gate") in (None, "PASS")` and does not require an arm canary record to exist. `oracle_ready` is set after A/A and nonleaking canary, but its `errors` and XL A/A completeness are not required. EMS `_gated` likewise permits missing XL and does not require the oracle canary's `errors == 0`.

Suggested VMS patch (insert before `vms_classify`, and use its booleans in `correctness_ok`; re-run final classification):

```
def vms_required_correctness(a):
    canary = a.get("canary")
    oracle = VMS.get("oracle_canary")
    want = int(a.get("level_wanted") or 16)
    is_dp = "--data-parallel-size" in [str(x) for x in a.get("flags", [])]
    require_xl = bool(_X) and not is_dp
    arm_canary_ok = (
        isinstance(canary, dict)
        and canary.get("level") == want
        and canary.get("errors") == 0
        and canary.get("foreign_leaks") == 0
    )
    oracle_canary_ok = (
        isinstance(oracle, dict)
        and oracle.get("errors") == 0
        and oracle.get("foreign_leaks") == 0
    )
    xl_ok = (a.get("xl_gate") == "PASS") if require_xl else True
    return arm_canary_ok and oracle_canary_ok and xl_ok

# In the `gate == "oracle"` branch, preserve oracle evidence:
VMS["oracle_canary"] = a["canary"]
# Set oracle_ready only after complete short/concurrent A/A,
# complete XL A/A when _X is supplied, and oracle canary error/leak checks.

# In vms_classify:
a["correctness_ok"] = (
    a.get("gate") == "PASS"
    and vms_required_correctness(a)
)
```

Do not apply the last assignment blindly to the baseline/oracle rows: baseline and oracle roles remain diagnostic. Keep DP2 XL explicitly `NOT_TESTED_BY_DESIGN`, not a fake PASS, and never promote a diagnostic DP2 row to long-context-certified without separate evidence.

Suggested EMS `_gated` changes:

```
oracle_canary = EMSR.get("canary_oracle")
if not isinstance(oracle_canary, dict):
    return False
if oracle_canary.get("errors") != 0 or oracle_canary.get("foreign_leaks") != 0:
    return False
if a.get("canary_leaks") != 0 or a.get("canary_errors") != 0:
    return False
if _X and a.get("xl_gate") != "PASS":
    return False
```

Additionally ensure `EMS_TGT_XL` is based on a COMPLETE A/A (using `_cmp_complete`), not just nonempty rows, and that `canary_oracle` is clean before marking the oracle certification-ready. An XL skipped for deadline should be reported `INCOMPLETE_XL`, not certified.

Validation: (a) XL = None, canary = None; (b) oracle canary `errors=4`; (c) XL A/A has an errored row; (d) arm canary errors = 1; (e) all required evidence valid. Cases a–d must be nonadoptable; e may be considered for adoption subject to other gates.

Fix 2 — No-first-token requests are not a clean throughput window

The window records `no_first_token` but `vms_window_healthy` never tests it. Correctness may be fine while timing evidence is incomplete, so classify as measurement invalid rather than claim the model output is corrupted.

```
# Add to vms_window_healthy conditions:
and w.get("no_first_token") == 0
and w.get("threads_not_joined") == 0
and w.get("idle_after") is True
```

Use `w.get("no_first_token") == 0` rather than `.get(..., 0)` to fail closed on missing instrumentation. **Test**: inject 3 no-first-token streams into a record with otherwise positive throughput and 100% running mean; `measurement_ok=False` is expected.

Fix 3 — Grid exception masks corruption

The current rule exempts *any* text containing >=30% digits, not just structured ARC grid output. The negative case `"1234567890"*5+"!"*64` includes 64 repeated exclamation marks but is classified healthy. Do not remove all grid exceptions: real grids are highly repetitive.

Minimal targeted fix: perform a 32-character repeated-character corruption test **before** allowing grid-like exemptions for non-grid fragments, and apply exemptions only to validated grid regions, not the entire response.

```
def text_bad_safely(x):
    s = (x or "").strip()
    if not s:
        return True
    if re.search(r"([^\s\d])\1{31,}", s):  # e.g. '!' x 32; keep repeated numeric grids valid
        return True
    # Current grid exception remains for structured grid-like output,
    # but do not use it as the sole proof of safe reasoning.
    return existing_text_bad(s)
```

This is a conservative detector improvement, not a complete semantic corruption classifier. Repeat the test against a legitimate all-zero ARC grid, JSON arrays, and ordinary reasoning. A benign coherent-but-wrong answer still requires task-level validation.

Fix 4 — INT4 MTP checkpoint static check should distinguish packed from proved dense

Current `out["ok"] = bool(mtp) and not not_ign` is vacuously true if `mods=[]`, even when **all** MTP tensors are packed. For this particular RedHat BF16-MTP checkpoint, packed MTP indicates the intended exemption

property was not established. The existing actual RedHat checkpoint has `re:^mtp.*` according to the supplied static evidence, so **this finding does not prove that the supplied checkpoint is bad**.

```
# Before out["ok"] assignment in vms_int4_mtp_static:
if not mtp:
    out.update(ok=False, why="No MTP weights found")
elif packed:
    out.update(ok=False, why="Expected BF16 MTP but packed MTP tensors found")
elif not mods:
    out.update(ok=False, why="Could not establish dense MTP linear-module exclusions")
elif not_ign:
    out.update(ok=False, why="Dense MTP linears missing from ignore list")
else:
    out.update(ok=True, why="MTP dense and all checked linears exempted")
```

This is a **checkpoint-specific strict guard** for the expected BF16 MTP layout. If mixed-quant MTP becomes intentional, use a separate explicit schema and weight-module mapping instead of banning packed MTP universally. **Test**: packed-only tensors, no ignore => fail; RedHat-style dense MTP with `re:^mtp.*` => pass; dense MTP with missing exemption => fail.

Fix 5 — Assert planned prompt counts

`vms_diverge` uses `expected=max(len(a),len(b))`. That verifies equality of two observed lengths, not equality to the design's planned counts. Normally `vms_greedy` returns one record per input, so this is mainly a forward-compatibility/data-integrity guard.

```
# In vms_judge, before comparison:
if any((len(s) != len(_S[:6]), len(c) != len(_S),
        len(base.get("serial_a", [])) != len(_S[:6]),
        len(base.get("conc_a", [])) != len(_S))):
    return "INCOMPLETE", {"gate_class": "INCOMPLETE", "why": "planned prompt row count mismatch"}

# In vms_judge_xl:
if len(x or []) != len(_X[:4]) or len(base.get("xl_a") or []) != len(_X[:4]):
    return {"verdict": "INCOMPLETE", "why": "XL row count mismatch"}
```

Test: a one-row oracle and one-row candidate cannot pass when 6 serial requests were planned. All valid full-size records must retain their previous classifications.

Fix 6 — A named PCIe-IPC winner needs runtime backend evidence

VMS `pcie_ipc.initialized=False` and EMS `pcie_ipc.workspace_built=False` are recorded, but neither adoption function checks them. That allows an arm labelled PCIe-IPC to pass with NCCL fallback. Do not count backend initialization as proof of GPU utilization; it is only a minimum gate, with a profiler trace needed for definitive attribution.

```
# VMS vms_classify:
wants_pcie = bool((a.get("env") or {}).get("VLLM_ALLREDUCE_USE_FLASHINFER_PCIE_IPC"))
backend_ok = (a.get("pcie_ipc") or {}).get("initialized") is True
if wants_pcie and not backend_ok:
    a["measurement_ok"] = False  # or `backend_verified=False`, `adoptable=False`

# EMS _gated:
if (a.get("env") or {}).get("SGLANG_ENABLE_PCIE_IPC_ALLREDUCE"):
    px = a.get("pcie_ipc") or {}
```

```
    if not (px.get("enabled") and px.get("workspace_built")):
        return False
```

IMPORTANT: EMS records its `env` as `SGLANG_ENABLE_PCIE_IPC_ALLREDUCE` and VMS its env as `VLLM_ALLREDUCE_USE_FLASHINFER_PCIE_IPC`. Verify that the SGLang 0.7.0.post1 overlay actually loads in the **worker** process and not just the initial import subprocess. **Test**: inject PCIe flag requested + initialization not observed; it must be classified as fallback/unverified, not as a verified PCIe speed winner.

Fix 7 — Distinguish BOUNDARY from production certification

The team deliberately permits `BOUNDARY` as an **adoptable benchmark candidate** because kernel/precision changes create numerical near-ties. That is a defensible *experiment policy*, but it is not a guarantee of correct ARC reasoning: plausible incorrect text may not trip a degeneration detector. Preserve the experiment's near-tie rule; do **not** silently rewrite it into an exact-ID requirement that rejects useful candidates.

Instead report separate fields:

```
candidate_for_arc_validation = bool(a.get("adoptable"))
certified_exact = candidate_for_arc_validation and a.get("gate_class") == "PASS_EXACT"
production_approved = certified_exact and bool(a.get("long_run_pass")) and bool(a.get("arc_task_accuracy_pass"))
# Boundary can become production_approved ONLY with an explicit separate
# task-level accuracy/long-run waiver recorded in the deployment decision.
```

This safeguards against treating a `BOUNDARY` leaderboard uplift as inherently safe while retaining candidate speed measurements.

Valid-arm false rejection and measurement fairness

- 20/24-**stream or XL windows**: `running_mean >= 0.8*level` can reject a healthy arm if finite 8192-token outputs naturally complete during the 75s window. Record whether each request reached `max_tokens`, the first-token timestamp, per-stream active duration and scheduled replacements. If many complete normally, either replenish to maintain load or label the band as `load_declined`, not model corruption.
- **Prefix caching and prefill**: 300 s settle+75 s measurement may exceed a 14-minute arm cap when prefill is very long. It correctly becomes mixed-load/INCOMPLETE, not a comparison-ready decode number. Keep raw `mixed_load` separate instead of deleting it.
- **XL matrix**: A missing XL due to deadline must not be reinterpreted as passed. Some DP/TP2 variants intentionally skip XL; keep these under `DIAGNOSTIC_NO_XL` rather than comparing them with XL-certified TP4 winners.
- **Factual budget**: SAFE and RISKY notebooks both independently launch their own reference and target-only oracle. Re-evaluate total cold-start costs for *three* separate notebook runs. The previous 260 min *per unsplit engine matrix* estimate is not automatically the total for the new split matrix.

Regression test summary

The following values were reproduced by extracting the actual function ASTs from the supplied builders (`builders/build_vms.py` V1/V2; `builders/build_ems.py` ARMS), compiling those functions and injecting synthetic inputs:

```
T1 VMS NO_XL_NO_CANARY ADOPTABLE= True
T2 VMS NO_FIRST_TOKEN_3 HEALTHY= True
T3 VMS GRID_HEURISTIC_PUNCTUATION_BAD= False
T4 VMS MTP_PACKED_NO_IGNORE STATIC_OK= True
T5 VMS 1_ROW_WHEN_PLANNED_6 VERDICT= PASS
T6 EMS NO_XL_ORACLE_CANARY_ERRORS GATED= True
T7 VMS PCIe_ARM_FALLBACK ADOPTABLE= True
```

Desired results after fixes: T1=False; T2=False; T3=True; T4=False (for BF16-MTP expectations); T5=INCOMPLETE; T6=False; T7=False. Also run *positive controls*: normal grids, complete A/A, error-free canaries, valid dense MTP, and actual initialized PCIe backend must not be wrongly rejected. **We did not modify or repackage the user's notebooks**; this is an audit and patch plan.

GO/NO-GO

- **GO**: Rev2f notebook SHA/JSON/syntax integrity; continued CPU-only repair and negative tests.
- **NO-GO**: Auto-adoption on the basis of current `adoptable=True` alone; unsupervised selection of PCIe-IPC winner; deployment of BOUNDARY winner without ARC task validation.
- **CONDITIONAL GO**: Bounded GPU smoke after R1, R2 and R6 are fixed and all seven negative tests plus positive controls pass. The first GPU session should validate real MTP memory pool, INT4 loader, backend selection, and target-only vs speculative correctness, before any full sweep.

Raw URLs for Claude (copyable, not hidden behind link labels)

All URLs below are direct upstream references already named in the supplied research files or inferred deterministically from the exact pinned repository path; they are **source pointers**, **not claims of having re-fetched the latest page in this audit**.

https://github.com/sgl-project/sglang/blob/v0.5.21/python/sglang/srt/speculative/eagle_worker_v2.py

https://github.com/sgl-project/sglang/issues/44145

https://github.com/sgl-project/sglang/pull/45151

https://github.com/sgl-project/sglang/issues/36548

https://github.com/sgl-project/sglang/issues/38009

https://github.com/sgl-project/sglang/issues/39597

https://github.com/sgl-project/sglang/issues/42917

https://github.com/sgl-project/sglang/pull/43029

https://github.com/vllm-project/vllm/issues/53912

https://github.com/vllm-project/vllm/issues/58807

https://github.com/vllm-project/vllm/pull/52182

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/v1/metrics/loggers.py

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/model_executor/models/qwen3_5_mtp.py

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/engine/arg_utils.py

https://github.com/vllm-project/vllm/blob/v0.32.0/vllm/model_executor/kernels/linear/**init**.py

Limits of this audit

The negative tests are function-level CPU reproductions, *not* complete Kaggle harness reruns nor genuine GPU kernel experiments. The supplied real GPU performance values belong to earlier E14 experiments, not rev2e/rev2f. No GPU work, Kaggle push, external web revalidation or independent end-to-end model run was performed during this audit. The exact safety decision for 60k–90k generated tokens remains open until verified on the candidate and production sampling settings.